

Cyber Yard

Cyber Yard jest projektem realizującym cyfrową implementację gry planszowej "Scotland Yard" wraz z wieloma dodatkowymi funkcjonalnościami. Gra zawiera wszystkie mechaniki obecne w oryginalnej grze oraz proceduralny generator mapy miasta, który umożliwia wytworzenie unikalnej losowej planszy. Rozgrywka na wygenerowanym mieście odbywa się w środowisku trójwymiarowym, które uwzględnia elementy ozdobne takie jak budynki i drzewa. Dostępna jest rozgrywka zarówno w trybie PvP typu "gorącego krzesła" jak i PvE. W ramach projektu zostało zaimplementowane łącznie 14 algorytmów sztucznej inteligencji, umożliwiających wybór przeciwnika o różnej strategii i poziomie trudności. Gra umożliwia również rozgrywkę online w ramach sieci LAN.

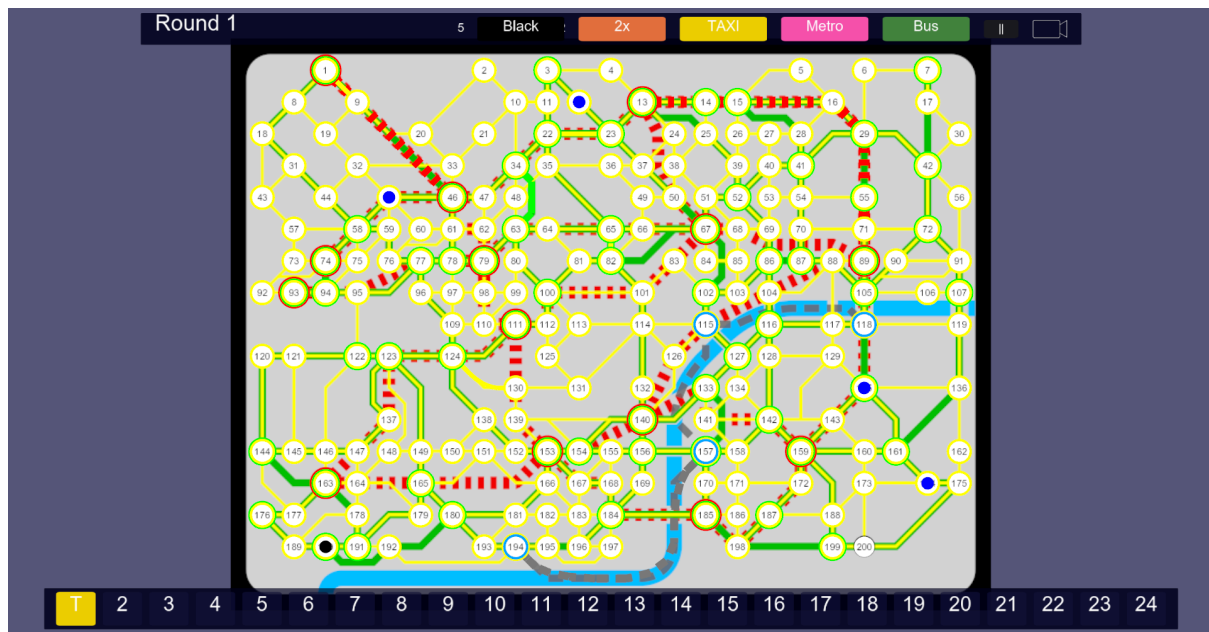
1. Zastosowane technologie

Główny program gry zrealizowano w języku C++, program odpowiedzialny za uczenie modeli AI napisano w języku Python. ZMQ pełni rolę API pomiędzy programem głównym a programem uczącym.

Warstwę graficzną programu głównego gry wykonano przy użyciu OpenGL oraz GLEW, kontekstem okna zarządza SDL2. Graficzne operacje matematyczne wykonywano z użyciem funkcji z biblioteki GLM. Za render glyphów tekstu odpowiada biblioteka FreeType. Ładowanie tekstur - STB Image. Parsowanie oraz serializacja danych JSON - nlohmann/json.

2. Rozgrywka na podstawowej planszy

Podstawowa warstwa rozgrywki opiera się na turowym systemie pościgu, realizowanym na grafie, który odwzorowuje miasto. Gracze przemieszczają się pomiędzy węzłami sieci, wykorzystując dedykowany system biletów (Taxi, Autobus, Metro), które determinują dostępne krawędzie ruchu. Zaimplementowana logika gry wprowadza kluczową asymetrię: podczas gdy pozycja detektywów jest jawna, Mr. X porusza się w ukryciu, a jego lokalizacja jest renderowana na planszy tylko w ściśle zdefiniowanych rundach (tzw. "show rounds"). Całość nadzorowana jest przez system, który automatycznie zarządza cyklem tur i weryfikuje warunki zwycięstwa: detektywi wygrywają poprzez zajęcie węzła zajmowanego przez Mr. X, natomiast Mr. X triumfuje po przetrwaniu określonej liczby rund lub w przypadku zablokowania ruchów przeciwników. Interfejs użytkownika wspiera ten proces, wizualizując potrzebne informacje, takie jak historię ruchów Mr. X, aktualny stan biletów czy ustawienia kamery.



Rys. 1 Rozgrywka w wersji podstawowej

3. Proceduralny generator map

Parki - generowane są wielokąty reprezentujące tereny zielone, z losową liczbą boków i rozmiarem.

Rzeka - przebieg rzeki tworzony jest na podstawie punktów kontrolnych i krzywych Beziera.

Sieć ulic - generowana jest hierarchiczna sieć ulic z rozróżnieniem na stopnie ważności (główne arterie, ulice drugorzędne, ścieżki w parkach itp.), zgodnie z założeniami artykułu naukowego.

Drzewa w parkach - w parkach rozmieszczane są drzewa w sposób losowy, z zachowaniem naturalnego zagęszczenia.

Dane o budynkach - na mapie pojawiają się budynki o proceduralnie generowanym położeniu, kształcie podstawy, wysokości, typie dachu i kolorystyce ścian/dachu. Budynki klasyfikowane są według typu (domy, sklepy, biura itp.). Budynki są generowane proceduralnie wzdłuż ulic na podstawie artykułu naukowego.

Mosty - generowane są w miejscach przecięcia ulic z rzeką.

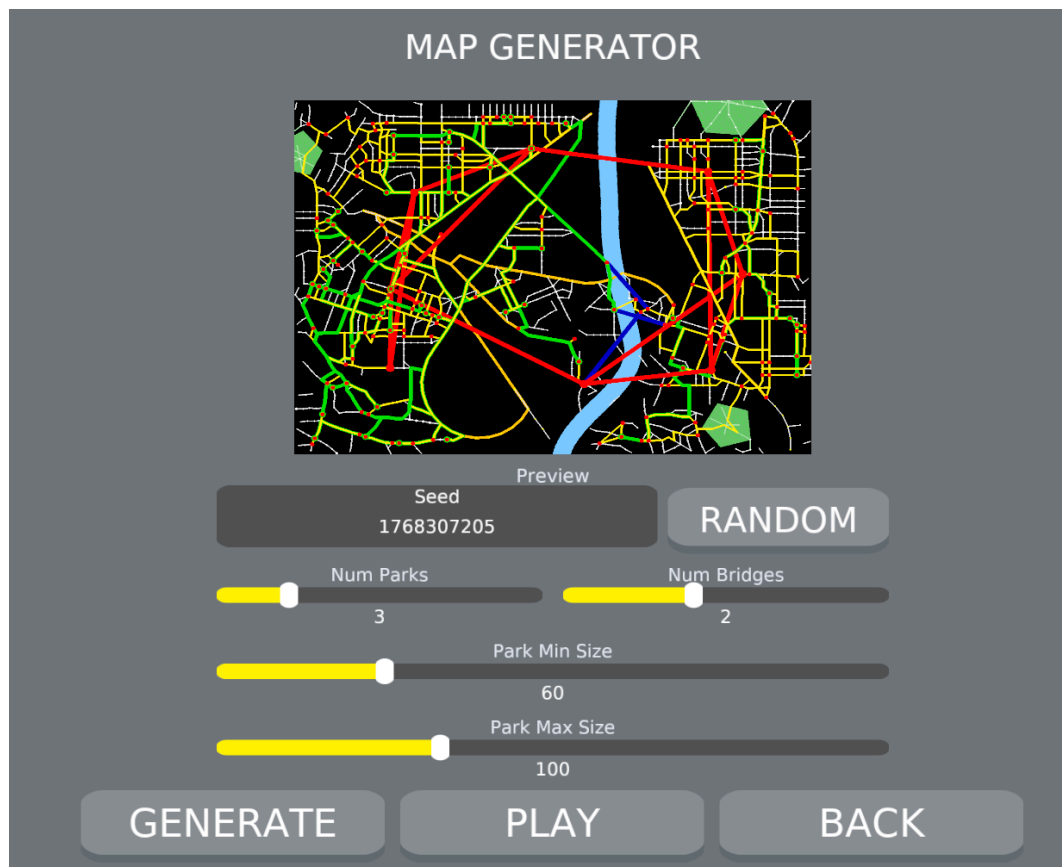
Stacje komunikacji - węzły grafu gry (punkty na skrzyżowaniach ulic) otrzymują połączenia różnych typów transportu:

- Taxi - najbardziej gęsta sieć, każdy węzeł łączy się z 2-6 najbliższymi sąsiadami (w parkach 2-4). Maksymalna odległość połączenia zależy od parametru gęstości.
- Autobus - rzadsza sieć, węzły poza parkami (parki obsługiwane tylko przez taxi) łączą się losowo z 1-3 dalszymi węzłami na większe dystanse niż taxi.
- Metro - wybierane jest ok. 25% węzłów spoza parków jako stacje metra, łączone w linie z rozgałęzieniami i połączeniami krzyżowymi.
- Prom - tworzona jest jedna linia promowa wzdłuż rzeki, łącząca węzły najbliższe równomiernie rozmieszczonym punktom na przebiegu rzeki (tylko węzły spoza parków).

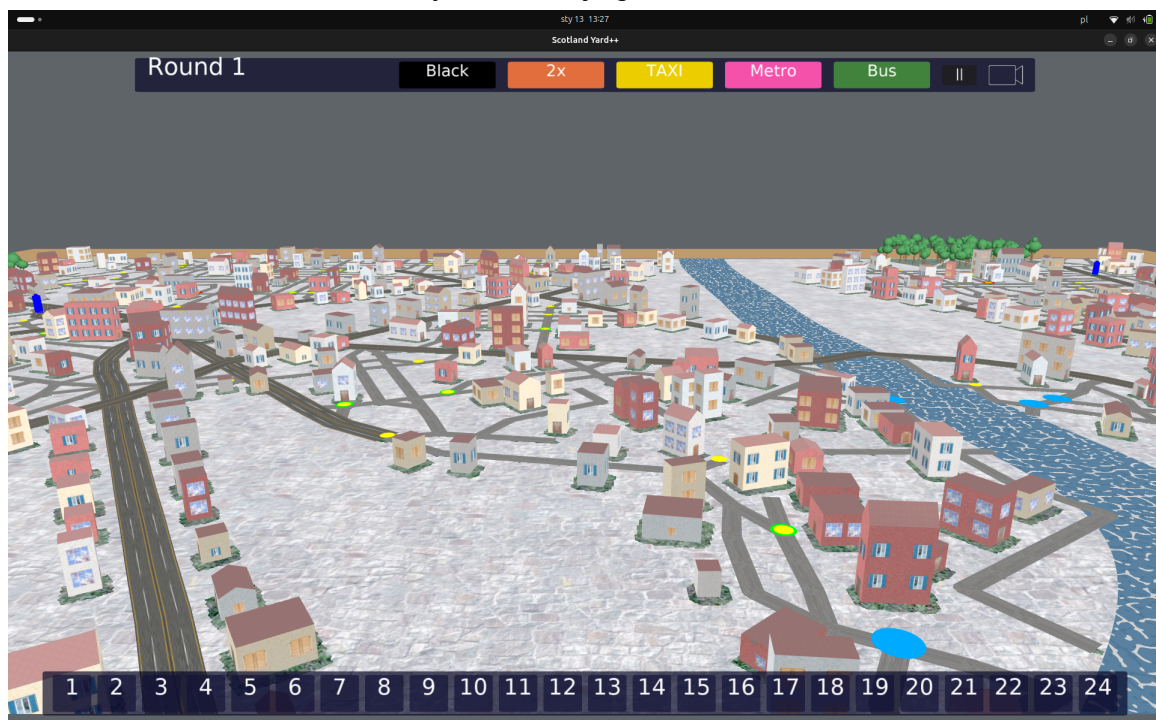
Idea generacji sieci ulic opiera się na artykule:

"Procedural Generation of Roads"

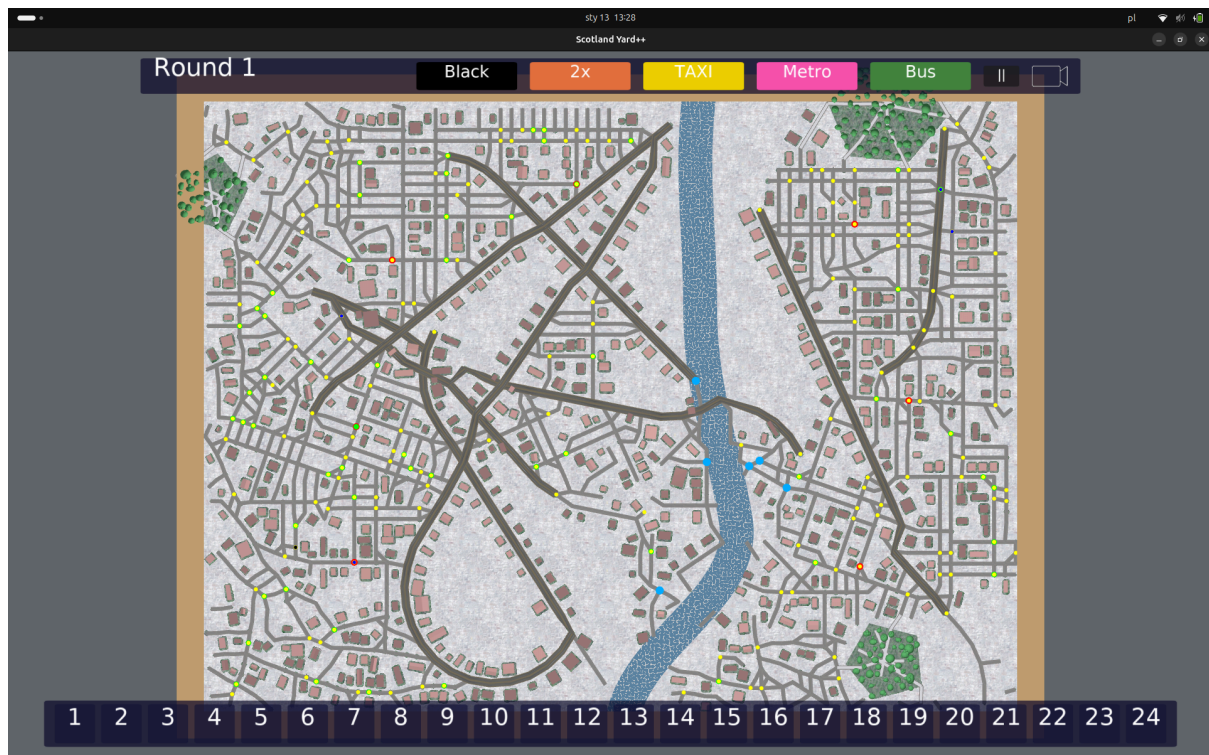
<https://www.cs.toronto.edu/~mackay/roadGeneration/roadGeneration.pdf>



Rys. 2 Interfejs generatora



Rys. 3 Widok wygenerowanej mapy pod kątem



Rys. 4 Widok z góry na przykładowo wygenerowaną mapę

4. Środowisko 3D

Środowisko graficzne gry to w pełni dynamiczna przestrzeń 3D, w której układ miasta, architektura oraz sieć transportowa są wynikiem działania algorytmów proceduralnych. Mapa składa się z kilku współzależnych warstw wizualnych, generowanych automatycznie przez silnik. W wizualizacji zastosowano elastyczną kamerę, która pozwala graczowi na płynne przechodzenie od obserwacji całego planu miasta z góry oraz do przybliżeń na konkretne detale grafu i budynków.

Warstwa podłoża

Fundamentem wizualizacji jest system strefowania, który automatycznie dostosowuje wygląd nawierzchni do przeznaczenia danego obszaru:

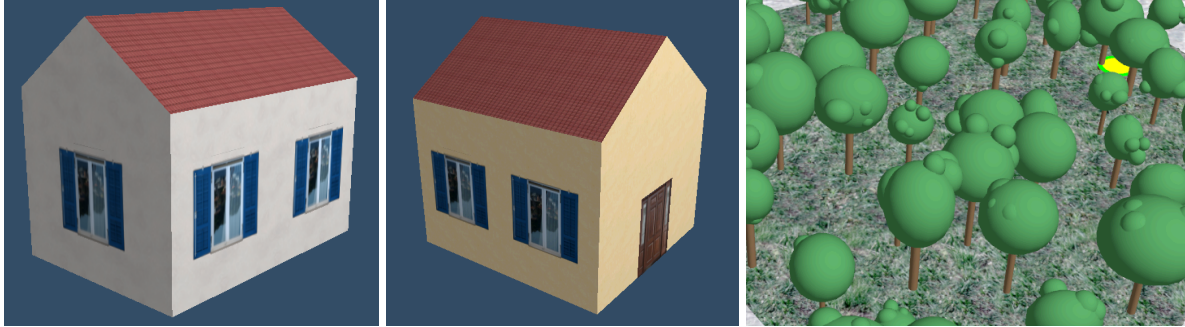
- Strefy miejskie wykorzystują tekstury chodników,
- Strefy zielone (parki) wykorzystują tekstury trawy,
- Rzeki przebiegające przez mapę reprezentowane są przez dedykowaną, animowaną taflę wody.



Rys. 5 Warstwa podłoża generowanej mapy.

Obiekty 3D

Na przygotowanym podłożu system rozmieszcza obiekty 3D, jakimi są modele budynków oraz drzewa. Modele budynków generowane są proceduralnie na podstawie obrysów fundamentów. Każdy budynek posiada indywidualnie dobraną wysokość, wariant dachu (płaski lub spadzisty) oraz ładowane tekstury fasad, okien i drzwi. Modele drzew rozmieszczane są na obszarach parków. Stanowią one proste modele, zbudowane z kul i walców.



Rys. 6 Obiekty 3D na generowanej mapie.

Infrastruktura Transportowa

System generuje sieć dróg, które podzielone są na klasy, co znajduje odzwierciedlenie w ich szerokości i zastosowanych teksturach. Wewnątrz parków generowane są dodatkowe ścieżki, również wyróżniające się teksturą.

Wizualizacja grafu i interakcja

Najwyższą warstwą środowiska jest graf połączeń, który nakłada logikę gry na świat 3D. Elementy, po których poruszają się pionki graczy, stanowią stacje, oznaczone markerami o kolorach oznaczających typ środka transportu, jakim można się poruszać z danego wierzchołka. Kolorem czerwonym oznaczono stacje metra, zielonym autobusu, a żółtym taxi.

Pionki w grze reprezentowane są jako modele 3D w kształcie walca, z oznaczeniem pionka mr X'a kolorem czerwonym, a pozostałych graczy w kolorze niebieskim. Nad pionkami, które jeszcze nie wykonały ruchu w danej rundzie, dodatkowo rysowane są półkule. Podczas ruchu, należy kliknąć pionek, którym gracz chce się poruszyć, wówczas na planszy, na stacjach do których może przejść, wyświetlają się oznaczenia w kształcie walca i w kolorze biletu, jakiego należy użyć, aby wykonać ruch. Klikając je, gracz się tam przemieści.

5. Algorytmy sztucznej inteligencji

W projekcie zastosowano heurystyczne algorytmy sterowania ruchem postaci Mr. X oraz policjantów:

Mr. X:

Algorytmy heurystyczne dla Mr. X głównie opierają się na maksymalizacji dystansu od najbliższego policjanta oraz minimalizacji prawdopodobieństwa wykrycia. Mr. X analizuje dostępne ruchy i wybiera ten, który prowadzi do najmniej przewidywalnej lokalizacji, często

korzystając z biletów specjalnych (np. czarnych) oraz tras o mniejszym natężeniu ruchu policyjnego.

Decoy movement - Mr. X generuje ruchy pozorne, które mają zmylić policjantów, sugerując fałszywe trasy ucieczki.

DFS (Depth-First Search) - Mr. X eksploruje możliwe ścieżki w głąb grafu, wybierając trasy prowadzące jak najdalej od policji.

Monte Carlo - Mr. X symuluje wiele losowych scenariuszy ruchu i wybiera ten, który statystycznie najczęściej prowadzi do uniknięcia złapania.

Policjanci:

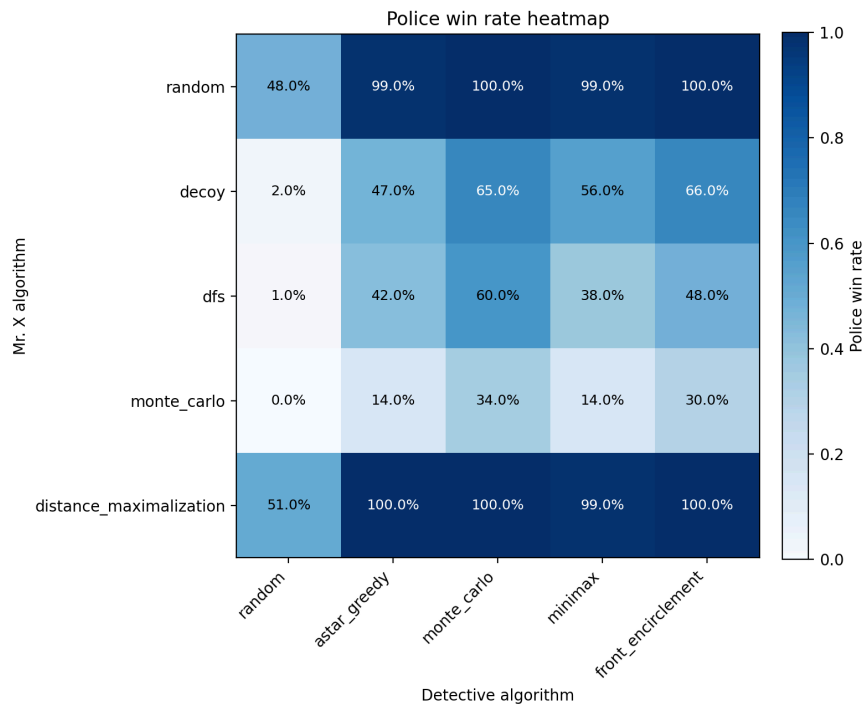
Policjanci stosują heurystykę minimalizacji dystansu do ostatnio znanej lub przewidywanej pozycji Mr. X. Każdy z nich wybiera ruch prowadzący najbliższej celu, przy czym algorytm uwzględnia także współpracę (np. blokowanie wyjść lub otaczanie podejrzanych obszarów). Heurystyka może być wspierana przez algorytm BFS lub A^* , aby efektywnie przeszukiwać graf miasta.

A^* - Policjanci wyznaczają najkrótszą ścieżkę do przewidywanej pozycji Mr. X, korzystając z heurystyki.

Front search encirclement - Policjanci tworzą linię poszukiwań, próbując otoczyć i zablokować Mr. X.

MiniMax - Policjanci analizują możliwe ruchy Mr. X i swoje, wybierając strategię minimalizującą maksymalne szanse ucieczki przeciwnika.

Monte Carlo - Policjanci symulują wiele potencjalnych scenariuszy ruchu Mr. X i wybierają działania, które najczęściej prowadzą do jego złapania.



Rys. 7. Porównanie skuteczności algorytmów heurystycznych każdy z każdym.

W projekcie stworzono 6 algorytmów AI opartych na uczeniu ze wzmocnieniem. PPO, MAPPO, SAC. Każdy zarówno dla Mr. Xa oraz policjantów.

PPO – Proximal Policy Optimization

Algorytm bezpośrednio optymalizuje politykę agenta. PPO stara się aktualizować politykę w sposób kontrolowany, aby nie wprowadzać zbyt dużych zmian w jednej iteracji. PPO używa klipu funkcji celu, aby ograniczyć zbyt duże kroki w przestrzeni polityki. Agent generuje doświadczenia w środowisku. Oblicza funkcję celu z klipem, np. ogranicza stosunek nowej i starej polityki. Optymalizuje politykę w kierunku maksymalizacji celu.

MAPPO – Multi-Agent Proximal Policy Optimization

Algorytm jest rozszerzeniem PPO dla środowisk wielu agentów. MAPPO stara się aktualizować polityki agentów w sposób kontrolowany, aby nie wprowadzać zbyt dużych zmian w jednej iteracji, przy jednoczesnym uwzględnieniu interakcji między agentami. Podczas centralized training funkcja wartości (critic) może korzystać z pełnej informacji o wszystkich agentach, a podczas decentralized execution każdy agent działa niezależnie. Agenci generują doświadczenia w środowisku, obliczają funkcję celu z klipem, podobnie jak w PPO, ale uwzględniając informacje innych agentów. Optymalizują swoje polityki w kierunku maksymalizacji wspólnego lub indywidualnego celu.

SAC – Soft Actor-Critic

Algorytm uczy agenta w sposób off-policy i stosuje podejście actor-critic z dodatkową maksymalizacją entropii polityki. SAC stara się aktualizować politykę tak, aby nie tylko maksymalizować nagrodę, ale także zachować odpowiednią różnorodność wyborów akcji (entropię), co poprawia eksplorację środowiska. Agent generuje doświadczenia, które są przechowywane w replay buffer, a następnie actor i critic uczą się na tych danych. Funkcja celu zawiera komponent nagrody i entropii, co pozwala stabilnie optymalizować politykę w środowiskach ciągłych. Optymalizacja polityki następuje w kierunku maksymalizacji skumulowanej nagrody wraz z zachowaniem wysokiej entropii działań.

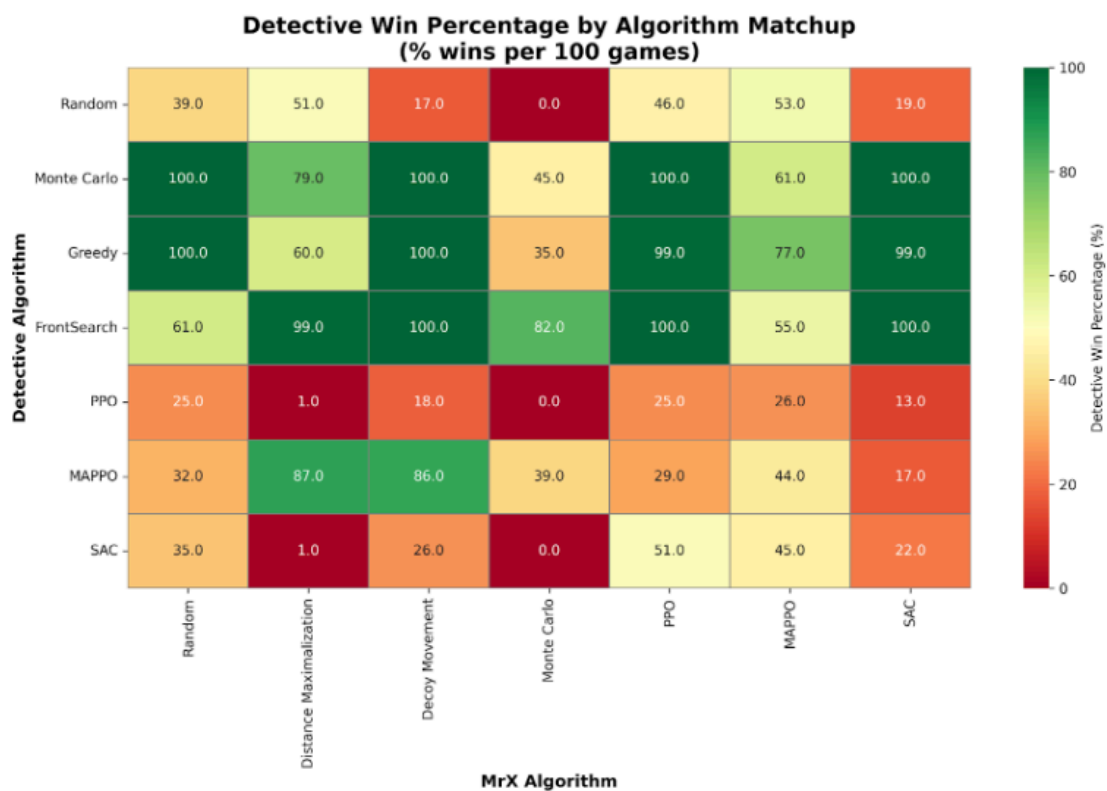
Aby uczyć każdy z algorytmów należało zdefiniować właściwy sposób przyznawania kar i nagród.

Dla Mr. Xa system przyznawał nagrodę analizując odległość w grafie od najbliższego policjanta, średnią odległość od wszystkich policjantów oraz liczbę połączeń wychodzących z węzła (o zagłębieniu dwóch węzłów) w którym się znajduje.

Dla policjantów jedynym wyznacznikiem była ich bezpośrednia odległość od rzeczywistego Mr. Xa, Im bliżej, tym wyższa nagroda.

Dodatkowo, zarówno algorytmy Mr. X-a i policjantów pod koniec rozgrywki dostawały bonus w postaci nagrody/kary, w zależności od tego czy wygrały czy przegrały.

Każdy z algorytmów rozegrał podczas uczenia około 50 000 rozgrywek (jeżeli uznamy, że średnio rozgrzywka posiada 12 ruchów, to algorytmy podjęły blisko 600 000 decyzji) zarówno z innymi modelami AI jak i algorytmami heurystycznymi.



Rys. 8. Porównanie skuteczności algorytmów heurystycznych i AI podczas rozgrywek każdy z każdym